



14. K-Nearest Neighbors

What if we want to predict data by learning its distance?

Previous Section:

 [13. Logistic Regression](#)

Contents:

[1. K-Nearest Neighbors - A Distance-Based Model](#)

[2. K-Nearest Neighbors - The Lazy Learner](#)

[3. K-Nearest Neighbors - Workflow](#)

[4. K-Nearest Neighbors - The Iris Dataset](#)

[5. K-Nearest Neighbors with Python](#)

[6. K-Nearest Neighbors for Regression](#)

[Summary](#)

[References](#)

This section moves on from linear and tree-based models to talk about something more intuitive → **K-Nearest Neighbor (or just KNN)**.

Instead of learning equations. Instead of building trees. KNN does something much more natural: It **looks around and finds the data closest to the predicted data**.

1. K-Nearest Neighbors - A Distance-Based Model

K-Nearest Neighbor (KNN) is a **supervised machine learning algorithm** used for classification, and sometimes regression, but at its core, it is most

commonly used as a **classifier**.

Instead of learning a model during training, KNN makes decisions **at prediction time**. Here's the idea: Given a new, unlabeled data point, KNN will:

- Look at the dataset
- Find the K **closest data points** (its neighbors)
- Check their labels
- And assign the **most common label** to the new point

That's it. No equations learned. No trees built. Just **distance** → **neighbors** → **majority vote**.

How does it measure "closeness"? The most common way is **Euclidean distance** — the straight-line distance between two points. But depending on the problem, other distance metrics can also be used (Manhattan, Minkowski, Chebyshev, etc.)

Once distances are computed:

- The nearest points are selected
- These neighbors are grouped by their class
- The class with the highest count wins

So the model is essentially asking: **"Among the closest data points... what do most of them look like?"**

The value of K is vastly critical. It directly affects how the model behaves:

- If K is **too small** → the model becomes sensitive to noise → **overfitting**
- If K is **too large** → the model becomes too smooth → **underfitting**

So K controls the balance between: Being **too reactive** and being **too generalized**

One critical detail I want you to remember is that KNN is heavily distance-based. That means: **Normalization is not optional — it is essential.**

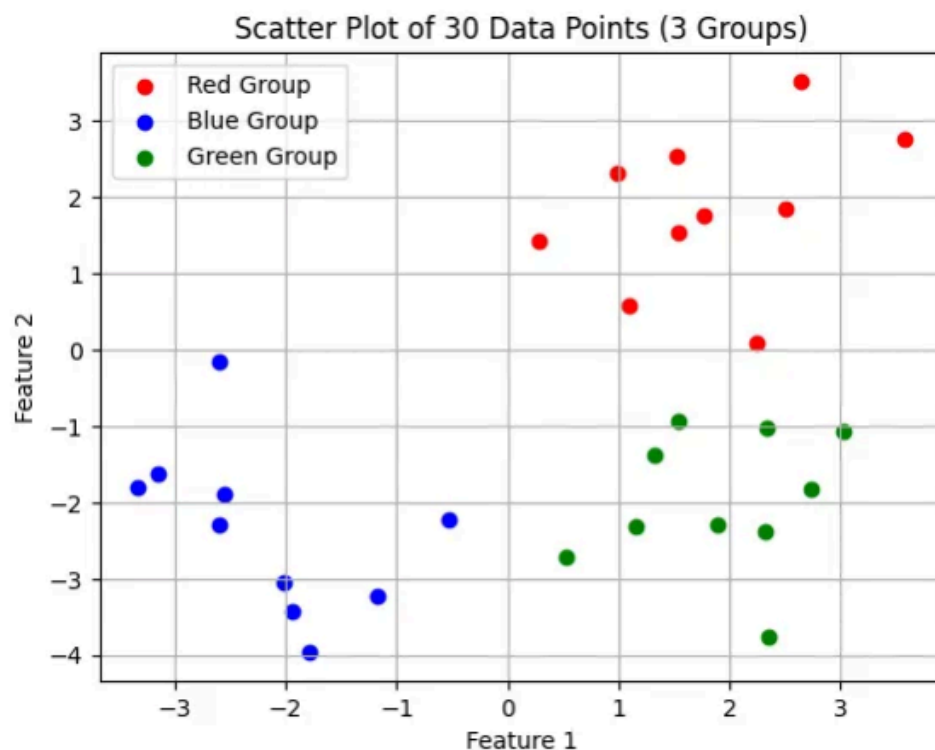
If features are on different scales (e.g., age vs salary), the distance calculation becomes biased.

The model will think one feature is more important simply because of its scale — not because it actually is.

KNN doesn't try to understand the data. It simply says: **"You are who your neighbors are."**; And sometimes that's more powerful than any equation.

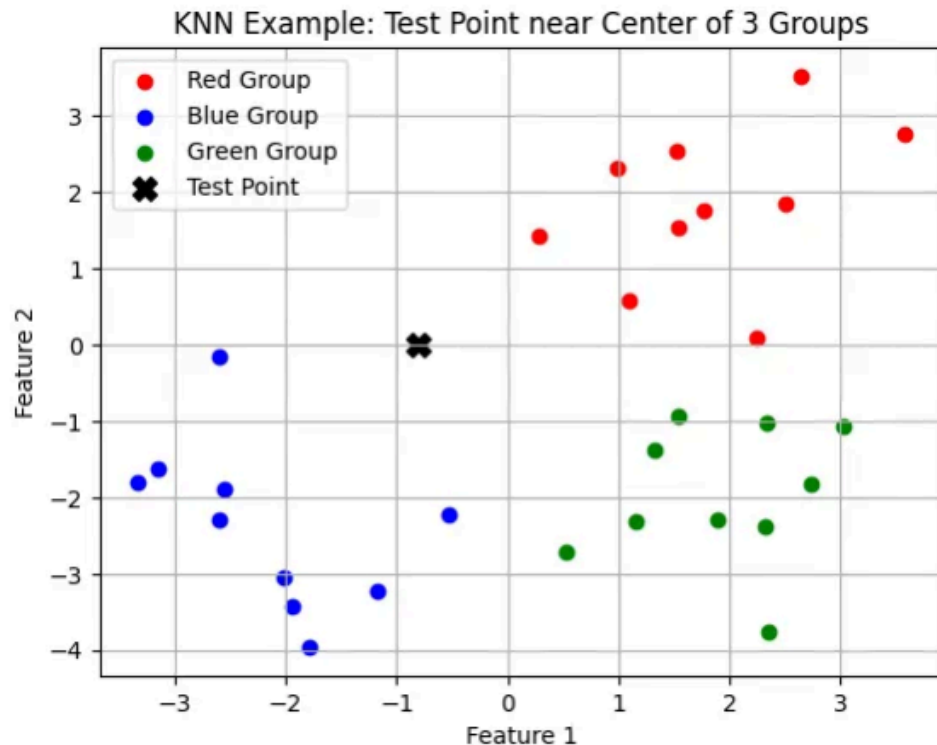
2. K-Nearest Neighbors - The Lazy Learner

Now let's make this more concrete. Imagine a plot with three clearly separated groups: **Red**, **Blue**, and **Green**.



Each group forms its own region — almost like clusters living in different parts of space.

And somewhere in this space, there is a **new point**. A point we don't know the label of yet.



The Question is now: Where does this point belong? Is it Red? Blue? Or Green?

How KNN answers this not by guessing. KNN doesn't build a formula. Instead, it does something very direct: It **looks around the point**.

- It finds the **K nearest data points** surrounding it
- Each of those neighbors already has a label (Red, Blue, or Green)
- Then it simply counts

And finally: The new point is assigned to the **majority class among its neighbors**

Now here's something interesting about KNN. It doesn't behave like the models we've seen before. It doesn't "learn" in the traditional sense. In fact, **KNN is what we call a lazy learner**.

What does "lazy" mean here? Looking back, most machine learning models follow this idea:

- Training phase → learn patterns, build a model
- Testing phase → use that model to predict

But KNN flips this idea completely.

- **Training phase (or perhaps not really training)** - When you “train” a KNN model. It doesn’t learn equations to build trees or optimize weights
 - Instead, it simply: **Stores the dataset (features X and labels Y)** → That’s it. No abstraction. No compression. No understanding. Just **memory**.
- **Testing phase — where everything happens:** This is where KNN actually starts working. When a new data point comes in:
 - It computes distances to **all stored data points**
 - Finds the **K nearest neighbors**
 - Looks at their labels
 - Assigns the class based on **majority voting**

So unlike other models: **KNN delays all computation until prediction time**

Why call it a “Lazy Learner”? Because it *procrastinates learning*. Instead of preparing during training. It waits until the very last moment — when a prediction is needed — and then does all the work.

Comparing the phases

- **Training phase: *No real learning* → *Just memorization of data***
- **Testing phase: *Heavy computation* → *Distance calculations* → *Neighbor selection* → *Final classification***

Other models are like students who: Study hard before the exam and then answer quickly

KNN is the opposite: It walks into the exam with **all the notes in hand**. And only when a question appears, it says: “*Wait... let me find the most similar questions I’ve seen before*”.

KNN doesn’t try to understand the world ahead of time. It simply remembers everything... and trusts that when the time comes. **The closest answers will guide it.**

As simple and intuitive as KNN is...There is one **very real problem** you will eventually run into: **Ties**.

A tie happens when the neighbors are **evenly split between classes**. For example: $K = 4$ and we get neighbors → 2 Red, 2 Blue

Now KNN is stuck. There is **no majority class**.

In theory, this is ambiguous. The model has no clear answer. But in practice — especially in implementations like **Scikit-learn** — the algorithm still has to return something.

So it applies a simple rule: **Pick the class that appears first**. And this is often determined by the internal ordering of labels, or typically **alphabetical order**.

This might create a problem:

- The prediction is no longer purely based on data
- It becomes dependent on **implementation details**
- Two identical situations could give biased results

In other words, the model is making a **forced decision**, not a confident one.

Ties are more likely when:

- K is **even** (like 2, 4, 6...)
- Classes are **balanced**
- Data points lie near **decision boundaries**

How do we deal with it? There are a few common strategies:

- Use an **odd value of K** (very common and simple fix)
- Use **distance-weighted voting** (closer neighbors matter more)
- Increase K to reduce instability

KNN trusts the crowd. But when the crowd is split. It doesn't know who to believe. So instead of thinking, it just picks the **first voice it hears**. And that's where intuition turns into **bias**.

3. K-Nearest Neighbors - Workflow

Up until now, we've talked about intuition. Now it's time to see how KNN actually **works step by step**. And you'll notice something interesting. There's no training logic here. Everything happens **during prediction**.

- **Step 1 — Choose K**

The very first decision: **How many neighbors should we look at?**

In our example → We choose $K = 13$. Why 13? It's large enough to reduce noise, and it's **odd**, which helps avoid ties

- **Step 2 — Compute distances**

Now we take the new data point. And compare it to **every single point in the dataset**.

We measure how far they are using distance. Most commonly, we use **Euclidean distance**:

$$d = \sqrt{(x_{new} - x_{old})^2 + (y_{new} - y_{old})^2}$$

That's for 2D. So we can visualize it

In higher dimensions, the idea extends naturally:

$$d = \sqrt{\sum_{i=1}^n (x_i^{new} - x_i^{old})^2}$$

Even if you can't visualize it. The math still works the same way.

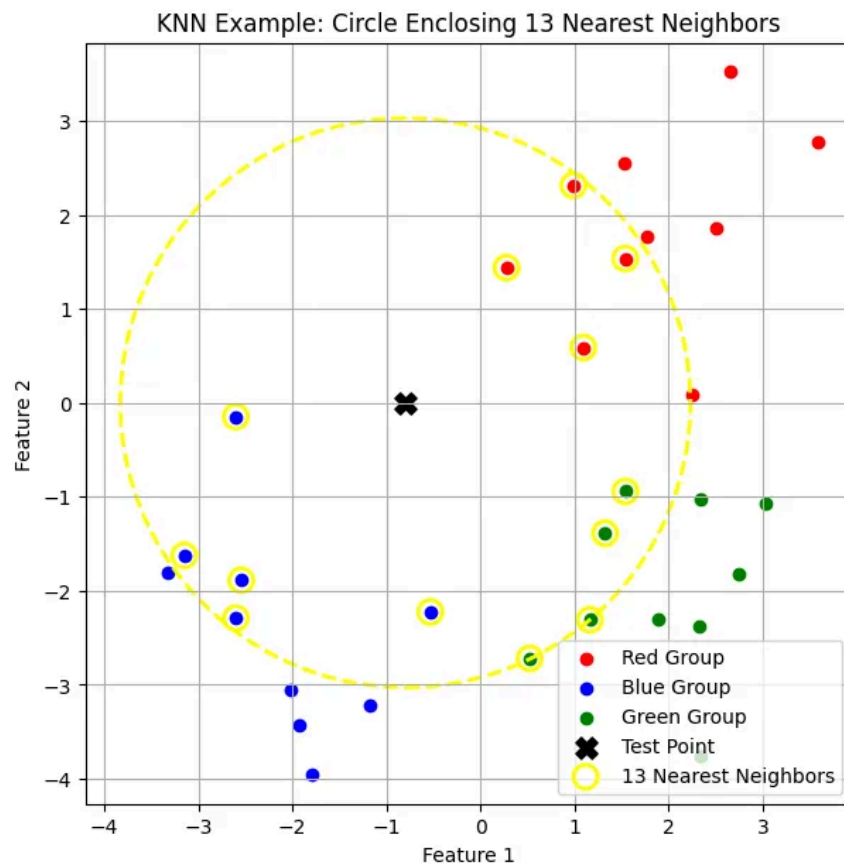
- **Step 3 — Find the nearest neighbors**

After computing all distances:

- Sort all points by distance (closest first)
- Select the K **smallest distances**

→ These are your K **nearest neighbors**

In our case: We take the **13 closest points**



- **Step 4 — Count the classes**

Now we look at those neighbors: 4 are **Red**; 5 are **Blue**; 4 are **Green**

- **Step 5 — Majority vote**

KNN now asks a very simple question: **Which class appears the most?**

Formally, we write:

$$\hat{y} = \text{mode}(y_1, y_2, \dots, y_K)$$

The answer: **Blue** appears the most (5 out of 13)

So the prediction is: **Final prediction: Blue**

- **KNN Recap (the full mechanism)** - Before assigning a label, KNN is essentially solving:

$$\text{neighbors} = \arg \min_{i \in \text{training data}} d(x_{\text{test}}, x_i)$$

Which means: Compute distance to every point → Find the closest ones → Let them decide

Then: Sort distances → Select K nearest neighbors → Apply majority voting
KNN is not complicated. It follows one simple philosophy:

Find what is closest... and trust it.

No model. No abstraction. Just distance, neighbors, and a vote.

4. K-Nearest Neighbors - The Iris Dataset

Now let's actually **walk through a real example**. And this dataset is called **Iris** (of course it is).

Sepal Length (cm)	Sepal Width (cm)	Petal Length (cm)	Petal Width (cm)	Target
5.1	3.5	1.4	0.2	0
4.9	3.0	1.4	0.2	0
4.7	3.2	1.3	0.2	0
4.6	3.1	1.5	0.2	0
5.0	3.6	1.4	0.2	0
5.4	3.9	1.7	0.4	0
4.6	3.4	1.4	0.3	0
5.0	3.4	1.5	0.2	0
4.4	2.9	1.4	0.2	0
4.9	3.1	1.5	0.1	0
5.4	3.7	1.5	0.2	0
4.8	3.4	1.6	0.2	0
7.0	3.2	4.7	1.4	1
6.4	3.2	4.5	1.5	1
6.9	3.1	4.9	1.5	1
5.5	2.3	4.0	1.3	1
6.5	2.8	4.6	1.5	1
5.7	2.8	4.5	1.3	1
6.3	3.3	4.7	1.6	1
4.9	2.4	3.3	1.0	1
5.0	3.5	2.0	1.0	?

- **The Setup:** We are given a dataset with 4 features: *Sepal Length*; *Sepal Width*; *Petal Length*; and *Petal Width*

Each row is a flower. And each flower belongs to a class: *Class 0* or *Class 1*

But the last row? That's our **test point**. It has features but no label. And our job is simple: **Use KNN to figure out what it is.**

- **Step 1 — Compare the test point to everything**

Since each data point is a **vector (4 features)**, we don't just look at one value. We compute the **distance across all features**.

For each training sample:

- Take the difference in each feature
- Square it
- Add everything together

→ This gives us the **distance sum**

(Notice something interesting: we don't even need the square root — the ranking stays the same.)

Distance Calculation Table (Squared Differences & Distances)

Δ Sepal Length²	Δ Sepal Width²	Δ Petal Length²	Δ Petal Width²	Distance Sum
0.01	0.00	0.36	0.64	1.01
0.01	0.25	0.36	0.64	1.26
0.09	0.09	0.49	0.64	1.31
0.16	0.16	0.25	0.64	1.21
0.00	0.01	0.36	0.64	1.01
0.16	0.16	0.09	0.36	0.77
0.16	0.01	0.36	0.49	1.02
0.00	0.01	0.25	0.64	0.90
0.36	0.36	0.36	0.64	1.72
0.01	0.16	0.25	0.81	1.23
0.16	0.04	0.25	0.64	1.09
0.04	0.01	0.16	0.64	0.85
4.00	0.09	7.29	0.16	11.54

1.96	0.09	6.25	0.25	8.55
3.61	0.16	8.41	0.25	12.43
0.25	1.44	4.00	0.09	5.78
2.25	0.49	6.76	0.25	9.75
0.49	0.49	6.25	0.09	7.32
1.69	0.04	7.29	0.36	9.38
0.01	1.21	1.69	0.00	2.91

- **Step 2 — Distances are computed**

After going through all points... We now have a list of distances.

- Some are small → very close
- Some are large → very far

For example:

- Smallest distance: 0.77
- Then: 0.85, 0.90, 1.01 . . .
- Much larger values appear for another class (like 8, 10, 12...)

→ Already, something is becoming clear.

- **Step 3 — Pick the nearest neighbors**

Now we sort all distances from smallest to largest. And we start testing different values of K :

- $K = 1$
 - Nearest distance: 0.77
 - Class: **0**
 - Prediction: **Class 0**
- $K = 3$
 - Distances: 0.77, 0.85, 0.90
 - All belong to Class 0
 - Prediction: **Class 0**
- $K = 5$

- Distances: 0.77, 0.85, 0.90, 1.01, 1.01

- Still dominated by Class 0

→ Prediction: **Class 0**

- **K = 7**

- Distances: 0.77, 0.85, 0.90, 1.01, 1.01, 1.02, 1.09

- Again... mostly Class 0

→ Prediction: **Class 0**

- **Step 4 — Final decision**

No matter how we expand the neighborhood. The closest points consistently belong to **Class 0**. So the final prediction is: **This flower belongs to Class 0.**

Look at the distance table carefully.

- Points from **Class 0** → very small distances
- Points from **Class 1** → very large distances

This means: The test point is **physically closer** to Class 0 in feature space. KNN didn't "learn" this. It didn't "understand" flowers. It simply observed:

┆ *"Everything closest to you... looks like Class 0."*

Even in higher dimensions. Even with real-world data. KNN still follows the same philosophy: **Closeness defines identity.**

And in this case. The answer was already there — hidden in the distances.

5. K-Nearest Neighbors with Python

Here's how you would build implement KNN in Python using the Scikit-Learn library:

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score

# Load the dataset
data = load_iris()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target, name='target')

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

# Train the model
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

# Optional: Evaluate
accuracy = model.score(X_test, y_test)
print("Accuracy:", accuracy)

```

Accuracy: 1.0

(KNN is usually considered the best model for the Iris dataset. Because of how heavily distance-based this data is)

6. K-Nearest Neighbors for Regression

Up until now, we've been using KNN for classification. But KNN can also be used for **regression**.

Instead of predicting a class label, the goal is now to predict a **continuous value** — something like price, temperature, or score. The idea is exactly the same: Find the *K* **nearest neighbors** based on distance.

The difference is in how the final prediction is made. Instead of taking a majority vote, KNN simply **averages the values** of the neighbors.

So instead of asking: *"Which class appears the most?"*. It asks: *"What is the average value of the closest points?"*

And just like before, everything depends on distance. Closer neighbors have more influence, especially if we use **distance-weighted averaging**. But the same limitations still apply: K must be chosen carefully, and normalization remains critical. KNN doesn't change its philosophy. It still believes: **"The closest points tell the truth."**

Summary

K-Nearest Neighbors is unlike most supervised learning models:

It doesn't learn equations. It doesn't build trees. It doesn't even "train" in the traditional sense. Instead, it simply remembers everything and waits.

Because of that, KNN is one of the **simplest algorithms** you will ever encounter.

And yet, it can be surprisingly **effective**. But here's the catch: **Its power depends entirely on when and how you use it.**

- When the data is well-structured → it works beautifully
- When distance truly reflects similarity → it shines
- When features are normalized and meaningful → it becomes reliable

But if these conditions are not met. KNN can struggle just as easily. KNN is not a model that tries to understand the world. It simply says: **"Show me what's closest... and I'll trust it"**. And if you place it in the right situation. That simple idea is more than enough.

References

<https://www.geeksforgeeks.org/machine-learning/k-nearest-neighbours/>

<https://www.ibm.com/think/topics/knn>

<https://www.youtube.com/watch?v=HVXime0nQeI>

Next Section:

 **15. K-Means Clustering**